

Multi-Agent Loan Default Prediction System

System Design Document

Architecture overview, agent responsibilities, and design rationale

Contents

1	Overview	1
1.1	Pipeline at a glance	1
1.2	Fusion formula	1
2	Agent responsibilities	1
2.1	Text Analyst (LLM, White Agent 1)	1
2.2	Feature Strategist (LLM, White Agent 2)	2
2.3	Subgroup Advocate (LLM, White Agent 3)	2
2.4	Green Agent (hub role)	3
2.5	Reporter (LLM, White Agent 4)	3
3	The Tier-A learned gating function	3
3.1	Motivation	3
3.2	Input features	4
3.3	Training objective	4
3.4	What the gate learned	4
3.5	Persistence	4
4	Worked example	5
4.1	Stage trace	5
4.2	Final verdict: NON-DEFAULT (0.253)	5
4.3	Which agents actually fired?	6
5	Design discussion: Strategist vs. Text Analyst overlap	6
5.1	The concern	6
5.2	What the Text Analyst loses by compressing	6
5.3	What the Strategist sees that nobody else does	7
5.4	Acknowledged risk of duplication	7
5.5	Validation test	7
5.6	Possible refinements (future work)	7
6	Evaluation methodology	7
6.1	AUPRC as primary discrimination metric	8
6.2	The six-question toolkit	8
6.3	Per-Agent KPI (MultiAgentBench §3.3)	8
7	Architectural decisions	8
7.1	Arbitrator merged into Green	8
7.2	Decision threshold unified to 0.5	9
7.3	MAX_SHIFT cap disabled in gate mode	9
7.4	Reflexion in Green	9

8	Open questions and future work	9
8.1	Strategist refinement	9
8.2	Text Analyst depth	10
8.3	Tier-A overfitting	10
8.4	Cost control	10
8.5	Topology experiments	10
9	Conclusion	10

1 Overview

The system predicts loan default for Grade C–G LendingClub-style loans by fusing a numeric XGBoost baseline with text-derived risk signals extracted from the borrower’s free-text description. Five LLM-based agents and one learned per-row gating function collaborate to decide *when* and *how strongly* to trust the text.

1.1 Pipeline at a glance

1. **Green — XGBoost Baseline** (numeric model, always fires)
2. **Text Analyst** (LLM, always fires if description present)
3. **Tier-A learned gate** (per-row text-weight, always fires)
4. **Feature Strategist** (LLM, conditionally fires)
5. **Subgroup Advocate** (LLM, fires only on Strategist deviation)
6. **Green — Arbitration** (LLM, fires only on Advocate veto)
7. **Fusion** (logit-space math)
8. **Green — Final Authority** (hard-rule flip gate)
9. **Green — LLM Review** (LLM, fires only on flip block)
10. **Reporter** (LLM, generates explanation)

The Green Agent owns multiple roles: baseline scoring, fusion validation, arbitration on veto, the Tier-A gate training, the final-authority flip gate, and the terminal LLM review. The previously-separate Arbitrator agent has been merged into Green (see §7.1).

1.2 Fusion formula

All probabilities are fused in logit space:

$$\begin{aligned}\text{fused_logit} &= \text{logit}(p_{\text{base}}) + w \cdot c \cdot (\text{logit}(p_{\text{text}}) - \text{logit}(0.5)) \\ \text{fused_pred} &= \sigma(\text{fused_logit})\end{aligned}$$

where p_{base} is the XGBoost baseline probability, p_{text} is the Text Analyst’s risk score, c is its confidence, and w is the text weight produced by the Tier-A gate (or by the static 5-scalar policy if no gate is loaded).

2 Agent responsibilities

2.1 Text Analyst (LLM, White Agent 1)

Input: borrower description text (truncated to 500 chars).

Output: `text_risk_score` $\in [0.05, 0.95]$ and `confidence` $\in [0.4, 0.95]$, plus a list of identified `risk_signals` and `protective_signals`, and a short `reasoning` sentence.

Scoring rubric (in the LLM prompt): start at 0.45, then add or subtract:

- -0.15 stable employment or specific job title
- -0.10 concrete repayment plan
- -0.10 “no missed payments”
- -0.08 specific income amount
- -0.05 single clear purpose
- $+0.15$ urgent or desperate language
- $+0.12$ behind on payments
- $+0.10$ unemployed
- $+0.08$ multiple problem debts
- $+0.05$ vague about income

A confidence cap is applied based on description quality (extremely short text caps at 0.45; rich detail caps at 0.95).

Role: first-pass distillation of prose into structured numbers. Designed for scalability (runs on every loan).

2.2 Feature Strategist (LLM, White Agent 2)

Input: description text, baseline probability, text risk and confidence, the gate’s per-row weight (or static grade weight), grade-level training history, and the previous iteration’s Advocate opportunity flags (in the training loop).

Output: a `strategy_type` (`keep_learned`, `case_specific_exception`, `fuzzy_only`, `explanation_only`, or `human_review`), a `proposed_weight`, optional constraints, a `rationale`, and (in the training notebook) a per-grade signal classification (`PERSISTENT_WIN` / `PERSISTENT_LOSS` / `NOISY` / `UNKNOWN`).

Trigger conditions:

- In all modes: $|\text{text_risk} - \text{baseline_p}| > 0.20$ (real disagreement)
- In static mode only: baseline in fuzzy zone $[0.30, 0.65]$, or extreme text with a mismatched grade weight

In gate mode, the fuzzy-zone trigger is dropped because the gate already incorporates `baseline_p` as a feature.

Role: *meta-judgment* about whether the gate’s weight needs case-specific correction based on description-level cues the gate cannot read (see §5 for the detailed analysis).

2.3 Subgroup Advocate (LLM, White Agent 3)

Input: previous and current fusion result (with per-grade AUC and AUPRC deltas), Green’s recent prediction trajectory, sample sizes per grade.

Output: `veto` (true/false), `affected_grades`, `critique`, `constraints`, `opportunity_flags`, and `reasoning`.

Decision principles (in the prompt):

- Veto only if harm is **persistent** (negative delta in ≥ 2 recent iterations) OR a single-step drop clearly exceeds sampling noise.
- Account for sample size: small n means high noise, so require a larger drop to veto.
- Flag **opportunities** (not vetoes): grades with consistently positive deltas worth nudging up.

- Be conservative about vetoing; false vetoes block real gains.

A rule-based fallback fires if the LLM call fails.

Role: subgroup safety check. In the demo, only fires when the Strategist proposes a non-trivial deviation from the baseline weight.

2.4 Green Agent (hub role)

Green plays seven distinct functions:

1. **Executor:** runs the XGBoost baseline, computes fused predictions, per-grade AUC *and* AUPRC.
2. **Validator:** checks if a proposed strategy passes (mean shift, flip-stat precision and recall) with *adaptive thresholds* derived from Reflexion experience.
3. **Reflexion planner:** before each training iteration, generates an *expectation* of Δ AUPRC, mean shift, and per-grade deltas; after evaluation, *reflects* on prediction error and accumulates `self.experience`.
4. **Arbitrator:** on Advocate veto, synthesizes the Strategist’s proposal, the Advocate’s constraints, and Green’s own experience into a compromise weight. (See §7.1.)
5. **Terminal decider:** `green_llm_review` when all candidates fail validation.
6. **Gate trainer:** learns the Tier-A per-row text-weight function via scipy L-BFGS-B (see §3).
7. **Final authority:** flip-gate hard rule before the verdict is issued (verdict reversals require high confidence + extreme text signal).

This concentration of roles is intentional: many of them share the same `self.experience` memory and `evaluate_fusion` computation, so splitting them would create awkward state-sharing.

2.5 Reporter (LLM, White Agent 4)

Input: the entire context (baseline, fused, weights, grade, text risk, confidence, the borrower’s key numeric features, the decision threshold).

Output: two short paragraphs in natural language describing (1) the numeric-risk picture and (2) the text’s role in shifting the prediction.

Role: human-readable summary. Reads only after fusion completes; never affects the prediction.

3 The Tier-A learned gating function

3.1 Motivation

The original system used five hand-tuned scalars `grade_weights` = {C: 0.03, D: 0.13, E: 0.08, F: 0.06, G: 0.03} to weight text influence per grade. This is dumb in two ways: (1) it does not vary within a grade, even though some cases within Grade C might benefit greatly from text while others should not; (2) it cannot respond to text content (a protective borrower’s note gets the same weight as an alarming one if they share a grade).

The Tier-A gate replaces these five scalars with a learned per-row function:

$$w_{\text{text}}(\text{row}) = w_{\text{max}} \cdot \sigma(\mathbf{x} \cdot \mathbf{w} + b)$$

where $\mathbf{x}$ is an 11-dimensional feature vector and the parameters $(\mathbf{w}, b)$ are optimized to minimize the log-loss of the fused prediction on the training set.

3.2 Input features

The gate sees:

- Five grade one-hot indicators (C, D, E, F, G)
- `baseline_p` (XGBoost output)
- `text_conf` (Text Analyst’s confidence)
- `text_risk` (Text Analyst’s score)
- Three derived features: $|\text{text_risk} - 0.5|$ (text extremity), $|\text{baseline_p} - 0.5|$ (numeric uncertainty), $|\text{text_risk} - \text{baseline_p}|$ (model disagreement)

Crucially, **the gate does not see the description prose**. It sees only the numeric distillations produced by Text Analyst.

3.3 Training objective

Let y_i be the binary default label, b_i the baseline probability, t_i the text risk score, c_i the text confidence, and $\mathbf{x}_i$ the standardized feature vector. Then for parameters $(\mathbf{w}, b)$:

$$w_i = w_{\max} \cdot \sigma(\mathbf{x}_i \cdot \mathbf{w} + b) \quad \hat{p}_i = \sigma(\text{logit}(b_i) + w_i c_i (\text{logit}(t_i) - \text{logit}(0.5)))$$

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] + \lambda \|\mathbf{w}\|^2$$

Optimized using L-BFGS-B from `scipy` with $\lambda = 10^{-3}$. The fusion formula is differentiable end-to-end through w_i .

3.4 What the gate learned

After training (on cached `subset_train` with $n = 880$), the dominant coefficients are:

Feature (post-standardization)	Coefficient
<code>text_risk</code>	−2.15
$ \text{text_risk} - 0.5 $	−0.40
$ \text{text_risk} - \text{baseline_p} $	+0.35
<code>text_conf</code>	+0.34
<code>baseline_p</code>	+0.13
bias	+0.36

The strongly negative `text_risk` coefficient means: the lower the text risk score (i.e., the more protective the text), the higher the gate-output weight. The gate has learned the asymmetry that *protective text deserves to be trusted more strongly than risky text* (the latter is likely already reflected in the numeric baseline).

3.5 Persistence

After training, the gate parameters $(\mathbf{w}, b, \boldsymbol{\mu}, \boldsymbol{\sigma}, \text{grades}, w_{\max})$ are serialized as plain Python lists/floats into the model pkl under key `gating_params`. The Streamlit demo loads this and reconstructs the per-row inference at run time. If the key is absent the demo falls back to the static 5-scalar policy.

4 Worked example

Input: A Grade C borrower with the following numeric profile and description:

Loan: \$14k at 17.5%, FICO 705–709, DTI 14%, revol_util 38%, ...
Description: “I am a registered nurse with 9 years of continuous employment at the same hospital. My annual salary is \$68,000. I have never missed a single payment on any account. This loan consolidates two low-balance credit cards.”

4.1 Stage trace

#	Stage	Computation	Output
1	Green Baseline	XGBoost on 16 numeric features	$p_{\text{base}} = 0.624$ (DEFAULT zone)
2	Text Analyst	LLM scores description; $0.45 - 0.15 - 0.10 - 0.05 \rightarrow 0.15$, clamp $\rightarrow 0.05$	$\text{text_risk} = 0.05, \text{conf} = 0.90$
3	Tier-A Gate	Apply learned function: dominant $\text{text_risk} = -2.15$ coefficient $\times$ low risk $\rightarrow$ high weight	$w_{\text{gate}} = 0.598$
4	Strategist	Triggered (score gap $0.574 > 0.20$). LLM reads prose, sees gate already captured everything. Decision framework: “keep_learned is the right default.”	$\text{keep_learned}, w = 0.598$
5	Advocate	<i>Skipped</i> — Strategist proposed no change	—
6	Green Arbitrate	<i>Skipped</i> — Advocate did not veto	—
7	Fusion	$\text{logit}(0.624) + 0.598 \cdot 0.90 \cdot (\text{logit}(0.05) - 0) = -1.082$	$\text{fused_pred} = 0.253$
8	Green Final Authority	Shift cap disabled (gate mode). Flip gate detects risky $\rightarrow$ safe; requires $\text{conf} \geq 0.87$ (got 0.90 ✓) and $ \text{text} - 0.5 \geq 0.35$ (got 0.45 ✓). Flip approved.	$\text{green_action} = \text{accepted}$
9	Green LLM Review	<i>Skipped</i> — no hard rule fired	—
10	Reporter	LLM generates explanation: text shifted probability from 0.624 to 0.253, changing verdict to NON-DEFAULT	2 paragraphs

4.2 Final verdict: NON-DEFAULT (0.253)

This example illustrates the system’s correct behavior: protective text with high confidence *does* get to flip a numeric verdict, but only when both the gate’s structural signal and Green’s outcome-level safeguards agree.

4.3 Which agents actually fired?

Agent	Fired?	Role on this case
Green Baseline	✓ always	gave 0.624
Text Analyst	✓ always (text present)	prose → (0.05, 0.90)
Tier-A Gate	✓ always	the actual decision-maker: 0.598
Strategist	✓ (score gap)	chose not to intervene — judgment
Advocate	— skipped	(no change to veto)
Green Arbitrate	— skipped	(no veto)
Fusion	✓ always	0.253
Green Final	✓ always	flip gate approved
Green LLM Review	— skipped	(no rule triggered)
Reporter	✓ always	explanation

The key observation: most cases do *not* require Strategist, Advocate, Arbitrate, or LLM Review. The Tier-A gate already produces per-case weights that work for the typical loan. The LLM-based conditional agents fire only when there is genuine signal disagreement or rule-level contention.

5 Design discussion: Strategist vs. Text Analyst overlap

5.1 The concern

A natural question: *both the Text Analyst and the Strategist read the description. Doesn't the Strategist duplicate the Analyst's work?*

The gate sees only the Analyst's compressed output (`text_risk`, `text_conf`) and never the original prose. The Strategist sees both — so if it is just re-scoring the same risk signals, it is in fact redundant.

5.2 What the Text Analyst loses by compressing

The Analyst follows a rule-based rubric (§2.1). This compression necessarily loses five categories of signal:

Signal type	Example	Analyst's handling
Verifiability	“Microsoft engineer 8 yrs” vs. “I have a good job”	Both trigger “−0.15 stable employment”
Internal contradiction	“Stable job 9 yrs, but currently behind on rent”	Mixed signals → middle score with low confidence
Detail specificity	“\$68K salary” vs. “decent income”	First triggers −0.08, second does not, but confidence may not reflect the gap
Life-event context	“divorce + stable income” vs. “new job + 30% raise”	Both might produce similar middle scores
“Too good to be true”	baseline= 0.7 (high risk) plus a perfect-record text	Analyst never sees the baseline, so cannot question consistency

The last row is the most architecturally significant: the Text Analyst operates in isolation from the numeric model. By construction, it cannot ask “does this text contradict what the numbers say?”

5.3 What the Strategist sees that nobody else does

	Text Analyst	Strategist
description prose	✓	✓
baseline_p (XGBoost output)	—	✓
text_risk, text_conf	(own output)	✓
gate’s per-row weight	—	✓
grade-level training history	—	✓

The Strategist’s task type is also different: *first-order* judgment (“what does the text say?”) vs. *second-order* meta-judgment (“how much should we trust it given the case context?”).

5.4 Acknowledged risk of duplication

If the Strategist’s prompt is poorly designed, it can collapse into a re-do of the Text Analyst. The current Strategist prompt counters this with an explicit decision framework:

“Read the borrower’s description prose. Does it contain CASE-SPECIFIC evidence — employer reputation, repayment-plan specifics, life-event context, urgency cues, contradictions with the numeric profile — that the gate’s structured features cannot see?”

The framework explicitly anchors the LLM’s role to “what the gate cannot see.” In gate mode, the default outcome is `keep_learned`; the Strategist must *actively justify* any deviation by citing a specific phrase from the description.

5.5 Validation test

A quick empirical check for duplication:

1. **Generic-signal case:** a description containing only signals the Analyst already covers. The Strategist should choose `keep_learned`; if it proposes a deviation, it is being redundant.
2. **Contradiction case:** “stable job 9 yrs” *and* “behind on rent.” The Strategist should propose `case_specific_exception` with the contradiction cited explicitly. If it just repeats the Analyst’s risk/protective list, it is duplicating.

5.6 Possible refinements (future work)

1. Hide (`text_risk`, `text_conf`) from the Strategist — force it to read prose only. *Risk:* now both agents do raw-text scoring, the real duplication.
2. Hide prose from the Strategist — give it only the structured numbers. *Risk:* it loses the case-specific detail that justifies its existence.
3. Restrict Strategist to a “contradiction auditor” role — only fires on internal inconsistencies the Analyst could not flag. *Trade-off:* cleanest division of labor, but limits the Strategist’s scope.

6 Evaluation methodology

The evaluation pipeline answers six concrete questions, each backed by a metric chosen to match the question, not by convention.

6.1 AUPRC as primary discrimination metric

AUC is the conventional choice but is misleading at imbalanced base rates (loan default is $\sim 15\text{--}25\%$ in C–G grades). AUPRC weights the positive class more honestly. All loop prints, decision logs, KPI milestones, Tier-A comparisons, and final test evaluations now lead with AUPRC; AUC is shown in parentheses as a secondary diagnostic.

6.2 The six-question toolkit

Question	Metric
Does fusion improve discrimination at this base rate?	AUPRC with bootstrap CI
Does fusion break calibration?	Brier score + ECE + per-bin reliability
Across plausible decision thresholds, is fusion net-beneficial?	Decision-curve net benefit
When fusion changes the verdict, does it reclassify correctly?	Categorical and continuous NRI
Is the LLM’s self-reported confidence informative?	LLM confidence reliability plot
When should we trust the text?	Selective fusion curve

The **selective fusion curve** is the headline measurement: for each candidate “trust score” (text confidence, model disagreement, or the Tier-A gate’s own output), we sort cases by that score and plot the fused ΔAUPRC as we sweep coverage from top-5% down to 100%. The curve’s shape *is* the answer to “when to trust the text”: monotonically decreasing means trust only at the top, inverted-U means there is an optimal coverage threshold, and a flat curve means the trust signal carries no useful information.

6.3 Per-Agent KPI (MultiAgentBench §3.3)

For per-agent attribution we adopt the milestone-based KPI from MultiAgentBench (Zhu et al., 2025):

$$\text{KPI}_j = \frac{n_j}{M}, \quad \text{KPI}_{\text{overall}} = \frac{1}{N} \sum_{j=1}^N \text{KPI}_j$$

where n_j is the number of milestones agent j contributed to and M is the total milestone count. We define 11 milestones — 7 per-iteration and 4 once-per-run — with explicit (deterministic, non-LLM) attribution rules. Each milestone names the contributing agents.

This surfaces the system’s *hub* structure: Green typically scores the highest KPI because it covers 7 distinct roles. Strategist and Advocate contribute to fewer milestones but in load-bearing places (the strategy-proposed and no-veto milestones, respectively).

7 Architectural decisions

7.1 Arbitrator merged into Green

The original architecture had Arbitrator as a separate White Agent. We merged it into Green for three reasons:

1. Arbitrator’s job (mediate Strategist vs. Advocate conflict) requires access to the same Reflexion experience and `evaluate_fusion` machinery that Green already owns.

2. Splitting it into a separate agent forced redundant state sharing and made the trajectory of past arbitrations awkward to retrieve.
3. Conceptually, arbitration is an authority decision, and Green is the system’s authority agent (final flip gate, terminal LLM review, baseline owner). All three roles are now consistent.

The LLM call and the prompt are essentially unchanged; only the *identity* attribution moved.

7.2 Decision threshold unified to 0.5

A prior version of the system had a calibrated threshold (0.62 in the training run) computed by `find_threshold` to enforce a precision- ≥ 0.40 floor. This was inconsistent with every other part of the system, all of which used 0.5 (per-grade precision/recall, flip stats, evaluation, etc.).

The calibrated threshold is now *informational only*: we still report “if you want precision ≥ 0.40 , threshold would need to be 0.62,” but the actual decision threshold is fixed at 0.5. This removes a confusing inconsistency without losing the diagnostic.

7.3 MAX_SHIFT cap disabled in gate mode

In static mode (5-scalar weights), Green enforces a maximum probability shift of 0.12 from baseline: a safety rail against text dominating the numeric model. This made sense when weights were dumb constants.

In gate mode, the gate has explicitly learned *when a large shift is appropriate*. Capping the shift at 0.12 directly contradicts what the gate learned. The shift cap is now disabled when `gating_params` is present in the pkl. The asymmetric flip-gate hard rule (verdict reversals require both high confidence and extreme text) remains active — but operates on outcomes rather than on weights, so it does not fight the gate.

7.4 Reflexion in Green

Green maintains `self.experience` across loop iterations. Before each evaluation, `generate_expectation` predicts the Δ AUPRC, mean shift, and per-grade deltas. After evaluation, `reflect` records the actual values and computes the prediction error. The accumulated history drives:

- **Adaptive validation thresholds:** max-mean-shift is derived from observed shift std rather than a fixed constant.
- **Memory-aware fallback:** `propose_counter_strategy` uses recent per-grade trends to adjust weights when LLM arbitration fails.
- **Trajectory injection into prompts:** both `green_llm_review` and `arbitrate` include the recent prediction trajectory as context, so the LLM can reason about systematic over/under-estimation.

8 Open questions and future work

8.1 Strategist refinement

The current Strategist is gate-aware and chooses `keep_learned` as the default, but its prompt does not formally enforce “find a contradiction or specific phrase, otherwise keep.” The `contradiction-auditor` refinement (§5.6) would make this clearer.

8.2 Text Analyst depth

The current Text Analyst uses a rule-based scoring rubric. A richer analyst that extracts structured fields (employer name, repayment plan specificity, internal contradictions) could reduce the Strategist’s overlap concern by shifting more of the prose interpretation upstream.

8.3 Tier-A overfitting

In the executed run, the Tier-A gate’s training log-loss improved by +0.030, but on the small test set ($n \approx 80$ cached observations) the gate’s test AUPRC (0.39) underperformed the static Strategist weights (0.43). The gate has 11 parameters and was trained on ~ 880 rows — borderline but not pathological. A larger Text Analyst cache would let us re-evaluate whether the gap is overfitting or test-set noise.

8.4 Cost control

The current configuration makes 5–6 LLM calls per training iteration (Strategist, Advocate, Green expectation, possibly Green arbitrate, possibly Green LLM review). With `max_iter=5` this is 25–30 calls per training run plus walkthroughs. An optional `llm_intensity` flag (`low` / `medium` / `high`) would let users trade off cost and decision quality.

8.5 Topology experiments

MultiAgentBench reports that graph-mesh topologies outperform chain and tree in research-style tasks. The current pipeline is essentially a chain (Strategist $\rightarrow$ Advocate $\rightarrow$ Green). A graph-mesh variant — where each White Agent independently sees the others’ outputs in parallel — would be an informative ablation, especially given Green’s existing hub role.

9 Conclusion

The system is a five-agent collaboration on loan-default prediction with a learned per-row gating function (Tier-A) at its core. The Green Agent owns the multiple authority roles — baseline, gate, arbitration, flip gate, terminal review — and uses Reflexion memory to adapt its validation thresholds and arbitration prompts across iterations.

The two architectural insights this document tries to convey:

1. **Most cases do not require LLM intervention beyond the Text Analyst and the Tier-A gate.** The Strategist, Advocate, Arbitrate, and LLM Review agents fire only when the gate’s structural signal might be inadequate. The system’s frugality with LLM calls is a feature, not a sign of underuse.
2. **The Strategist’s value is meta-judgment, not re-scoring.** The Text Analyst tells us what the text says; the Strategist tells us whether to trust that signal given the case’s full context. Whether this distinction holds up in practice is the subject of ongoing refinement (§5).